

Chapter 1

Methodology

1.1 Spatial Domain and Grid Representation

Let the spatial domain be discretized into latitude-longitude grid,

$$\{\phi_i\}_{i=1}^H, \{\lambda_j\}_{j=1}^W \quad (1.1.1)$$

where ϕ_i, λ_j denote the grid-point latitudes and longitudes, respectively.

The spatial domain is partitioned into rectangular grid cells,

$$C_{ij} \approx [\lambda_j, \lambda_{j+1}] \times [\phi_i, \phi_{i+1}], i = 1, \dots, H, j = 1, \dots, W \quad (1.1.2)$$

The corresponding spatial mesh is defined as:

$$(\Lambda, \Phi) = (\lambda_j, \phi_i). \quad (1.1.3)$$

1.2 Trend Analysis using Seasonal Mann-Kendall (SMK)

Test

This study employs the Seasonal Mann-Kendall (SMK) test to detect monotonic trends in monthly temperature and AOD series while explicitly considering seasonality, unlike its

classical counterpart (the Mann-Kendall Test), which treats the entire record as a single sequence, while the SMK test divides the data into “seasons” (in our case, months) and computes separate Kendall S statistics for each month across years. These S per-month values are then added to get an overall test statistic. This keeps the seasonal patterns without presuming any specific distribution for the data. The SMK test is robust even when there are missing values, which are prevalent in long-term atmospheric records, as it is rank-based (Hirsch et al., 1984).

Given the time series for our monthly data for each grid point, we have applied the SMK test. For each grid point and each variable time series dataset, SMK splits the data into m seasons (here $m = 12$), where season may be a month, week etc. and computes a separate Kendall statistic for each month across years. The detailed mathematics for the SMK test at one grid point and for one variable is as follows.

Mathematical Formulation

Let

$$X_i = \{x_{i1}, x_{i2}, x_{i3}, \dots, x_{in}\}, \quad (1.2.4)$$

denote the subseries of observations for month i over n years.

For each month i , compute the Mann–Kendall S-statistic:

$$S_i = \sum_{k=1}^{n_i-1} \sum_{j=k+1}^{n_i} \text{sgn}(x_{ik} - x_{ij}). \quad (1.2.5)$$

The sign function is defined as follows:

$$\text{sgn}(x_{ik} - x_{ij}) = \begin{cases} +1, & (x_{ik} - x_{ij}) > 0 \\ 0, & (x_{ik} - x_{ij}) = 0 \\ -1, & (x_{ik} - x_{ij}) < 0 \end{cases} \quad (1.2.6)$$

Under the null hypothesis of no trend, each S_i has an expectation $E[S_i] = 0$. When ties occur, let t_{ip} be the length of the p^{th} tied group i^{th} month, and let g_i be the total number of

tied groups for i^{th} month. Then the variance of S_i is:

$$\text{Var}(S_i) = \frac{1}{18} \left[n_i(n_i - 1)(2n_i + 5) - \sum_{p=1}^{g_i} t_{ip}(t_{ip} - 1)(2t_{ip} + 5) \right] \quad (1.2.7)$$

The overall SMK statistic is calculated by summing the monthly values represented by S_i :

$$S_{SMK} = \sum_{i=1}^{12} S_i \quad (1.2.8)$$

With total variance:

$$\text{Var}(S_{SMK}) = \sum_{i=1}^{12} \text{Var}(S_i) \quad (1.2.9)$$

For sufficiently long records ($n \geq 3$), S_{SMK} is approximately normal. We compute:

$$Z = \begin{cases} \frac{S_{SMK}-1}{\sqrt{\text{Var}(S_{SMK})}}, & S_{SMK} > 0 \\ 0, & S_{SMK} = 0 \\ \frac{S_{SMK}+1}{\sqrt{\text{Var}(S_{SMK})}}, & S_{SMK} < 0 \end{cases} \quad (1.2.10)$$

1.3 Formation of Relative Humidity Data

Relative humidity (RH) is defined as the ratio of actual vapour pressure (e) to saturation vapour pressure e_s .

Therefore, we first calculated the saturation vapour pressure using the Magnus-Tetens approximation (Alduchov et al., 1996), which is robust for the temperature range observed in the NWH region.

$$e_s(T) = 6.112 \times \exp\left(\frac{17.67 \times T}{T + 243.5}\right) \quad (13)$$

Where T is the temperature in $^{\circ}\text{C}$ and $e_s(T)$ is in hectopascals (hPa).

Subsequently, the actual vapour pressure (e) was derived from the specific humidity (q , kg/kg) and surface pressure (p , hPa), which is defined as follows:

$$e = \frac{q \times p}{\epsilon + (1 - \epsilon)q} \quad (14)$$

Where $\epsilon \approx 0.622$ represents the ratio of the molecular weight of water vapour to dry air.

Finally, RH was computed by dividing eq. (10) and eq. (11) as:

$$\text{RH (\%)} = \frac{e}{e_s(T)} \times 100 \quad (15)$$

1.4 Framework for Seasonal Scalar and Wind Field Visualization

The variable like AOD, and wind field shows significant spatial heterogeneity, and seasonal variation due to atmospheric dynamics, emission sources and region topography. To systematically analyse these patterns, the data are represented on a discretized latitude-longitude grid and aggregated over climatologically meaningful seasonal intervals.

1.4.1 Season Mean Scalar Field:

Let $X(\phi, \lambda, t)$ represent a scalar variable (e.g., AOD) at time t . For a given season (winter, pre-monsoon, monsoon, post-monsoon) comprising T time steps, the seasonal climatological mean is calculated as:

$$F_{ij} = \frac{1}{T} \sum_{t \in \text{season}} X(\phi, \lambda, t) \quad (16)$$

The resultant scalar field is presumed to be piecewise constant within each grid cell:

$$F(\phi, \lambda)|_{C_{ij}} \approx F_{ij} \quad (17)$$

which aligns with nearest-neighbor or cell-averaged visualization methodologies generally employed in gridded atmospheric datasets.

1.4.2 Seasonal Mean Wind Field:

Let $U(\phi, \lambda, t)$ and $V(\phi, \lambda, t)$ represent the zonal and meridional wind components, respectively. The seasonal average wind components are defined as:

$$U_{ij} = \frac{1}{T} \sum_{t \in \text{season}} U(\phi, \lambda, t), \quad V_{ij} = \frac{1}{T} \sum_{t \in \text{season}} V(\phi, \lambda, t) \quad (18)$$

Therefore, the resulting vector wind field is:

$$\mathbf{W}_{ij} = (U_{ij}, V_{ij}) \in \mathbb{R}^2 \quad (19)$$

1.4.3 Wind Speed and Visual Scaling:

The wind speed magnitude at each grid point is calculated using the Euclidean norm:

$$s_{ij} = \|\mathbf{W}_{ij}\|_2 = \sqrt{U_{ij}^2 + V_{ij}^2} \quad (20)$$

To highlight areas of stronger flow in the streamline depiction, the local linewidth is set to:

$$\text{lw}_{ij} = \alpha \frac{s_{ij}}{\max_{i,j} s_{ij}}, \quad \alpha \approx 2 \quad (21)$$

1.5 Relative Humidity Field

Let $\text{RH}(\phi, \lambda, t)$ represent a RH at location (ϕ, λ) and time t .

Seasonal Mean RH field:

The average relative humidity for season s at grid node (ϕ_i, λ_j) is defined as:

$$\text{RH}_s(i, j) = \frac{1}{T} \sum_{t \in \text{season}} \text{RH}(\phi_i, \lambda_j, t), \quad i = 1, \dots, H, \quad j = 1, \dots, W \quad (22)$$

The resulting field satisfies

$$\text{RH}_s \in \mathbb{R}^{H \times W} \quad (23)$$

Threshold Contour:

The 60% relative humidity contour is defined as the level set

$$\Gamma_{60}^s = \{(\phi, \lambda) : \text{RH}_s(\phi, \lambda) = 60\} \quad (24)$$

1.6 Spatio-Temporal AI Framework

1.6.1 Introduction

This study develops a deep learning model that forecasts temperature from aerosol optical depth measurements. The model processes sequences of spatial data to predict future temperature patterns. This approach combines three key components: a feature extraction network, a spatial processing module, and a temporal learning system.

The model takes sequences of aerosol measurements as input. Each measurement represents a 19 X 19 grid covering the study region. The model learns to map these aerosol patterns to temperature distributions at future time points. This prediction task requires the model to capture both spatial relationships within each time step and temporal patterns across the sequence.

The methodology addresses three main challenges. First, the model must extract meaningful features from aerosol data that relate to temperature patterns. Second, it must understand how temperature varies across space while respecting atmospheric physics. Third, it must learn temporal dependencies that govern how aerosol conditions influence future temperature.

1.6.2 Data Preparation and Preprocessing

1.6.2.1 Input Data Structure

The input data consists of three aerosol optical depth channels. These channels measure black carbon aerosol optical depth, sulfate aerosol optical depth, and dust aerosol optical depth for particles smaller than 2.5 micrometers. Each channel provides a different view of atmospheric aerosol content. Black carbon primarily comes from combustion processes. Sulfate aerosols form from industrial emissions and natural sources. Dust aerosols originate

from soil and desert regions.

The model receives these three channels at each time step. A sequence length of six time steps provides enough temporal context for the model to learn meaningful patterns. This means the model sees six consecutive aerosol measurements before making a prediction.

1.6.2.2 Temporal Encoding

Atmospheric patterns follow seasonal cycles. To help the model recognize these cycles, the data includes temporal encoding features. These features use sine and cosine functions to represent the time of year.

The encoding works as follows. Each time step corresponds to a month in the calendar year. The month number converts to an angle that ranges from zero to two pi radians. This angle captures the circular nature of seasons. January comes after December, so the encoding must reflect this continuity.

Two channels encode this temporal information. The first channel contains the sine of the angle. The second channel contains the cosine. Together, these channels give the model a clear signal about seasonal position. The sine and cosine values repeat annually, which matches the seasonal cycle of atmospheric conditions.

1.6.2.3 Autoregressive Features

Temperature at the current time relates to temperature at the previous time. This autoregressive property appears in many atmospheric processes. The model uses this information by including temperature from the previous time step as an input feature.

This feature gives the model a reference point. When predicting temperature at time t , the model knows the temperature at time t minus one. This information helps the model learn temperature changes rather than absolute values. Learning changes proves easier than learning absolute predictions from scratch.

1.6.2.4 Data Normalization

Raw aerosol and temperature values span different ranges. Aerosol optical depth typically ranges from zero to several units. Temperature values depend on the region and season.

These different scales can cause training problems.

The normalization process standardizes each variable. For the aerosol channels, the method computes the mean and standard deviation from the training data. Each value transforms by subtracting the mean and dividing by the standard deviation. This transformation creates features with mean zero and standard deviation one.

The temperature channel receives the same treatment. The normalization uses only training data statistics. This prevents information from the test set from influencing the training process. Using test set statistics would create data leakage and invalidate the evaluation.

The normalization parameters come from spatial and temporal averaging. For each channel, the method computes one mean value and one standard deviation value. These values apply to all spatial locations and time points within that channel.

Mathematical Formulation

Input: A sequence of T timesteps with enhanced feature representation:

$$\{\mathbf{X}_1, \mathbf{X}_2, \dots, \mathbf{X}_T\} \quad \text{where} \quad \mathbf{X}_t \in \mathbb{R}^{B \times C \times 19 \times 19} \quad (1.6.11)$$

where:

- B : batch size
- $C = 3 + 1 + 2 = 6$ channels:
 - 3 AOD channels: BC AOD, SO_4 AOD, DU AOD
 - 1 Autoregressive temperature channel: T_{t-1}
 - 2 temporal encoding channels: $\sin(\theta_t), \cos(\theta_t)$

Temporal Encoding:

The temporal angle encodes monthly cyclicity:

$$\theta_t = \frac{2\pi \cdot (t \bmod 12)}{12} \quad (1.6.12)$$

Two constant spatial channels are created for each timestep:

$$\mathbf{X}_t^{\text{time}} = \begin{bmatrix} \sin(\theta_t) \cdot \mathbf{1}_{19 \times 19} \\ \cos(\theta_t) \cdot \mathbf{1}_{19 \times 19} \end{bmatrix} \quad (1.6.13)$$

where $\mathbf{1}_{19 \times 19}$ is a matrix of ones. This encoding ensures December and January are close in feature space despite being 11 months apart numerically.

1.6.3 Model Architecture

The model architecture follows a hierarchical design. Each component handles a specific aspect of the learning problem. The first component extracts local features from each input channel. The second component builds spatial understanding across multiple scales. The third component learns temporal patterns from the sequence. The final component generates the temperature prediction.

This separation of concerns makes the model easier to understand and debug. Each component can learn its task without interference from the others. The components work together in sequence, passing information forward through the network.

The overall **Architecture Overview** of proposed framework is shown in Figure ?? . The complete pipeline with sequence-to-sequence prediction:

$$\hat{\mathbf{Y}}_{1:T} = \mathcal{F}_{\text{FC}} \circ \mathcal{F}_{\text{LSTM}} \circ \{\mathcal{F}_{\text{UNet}} \circ \mathcal{F}_{\text{MKB}}\}_{t=1}^T (\{\mathbf{X}_1, \dots, \mathbf{X}_T\}) \quad (1.6.14)$$

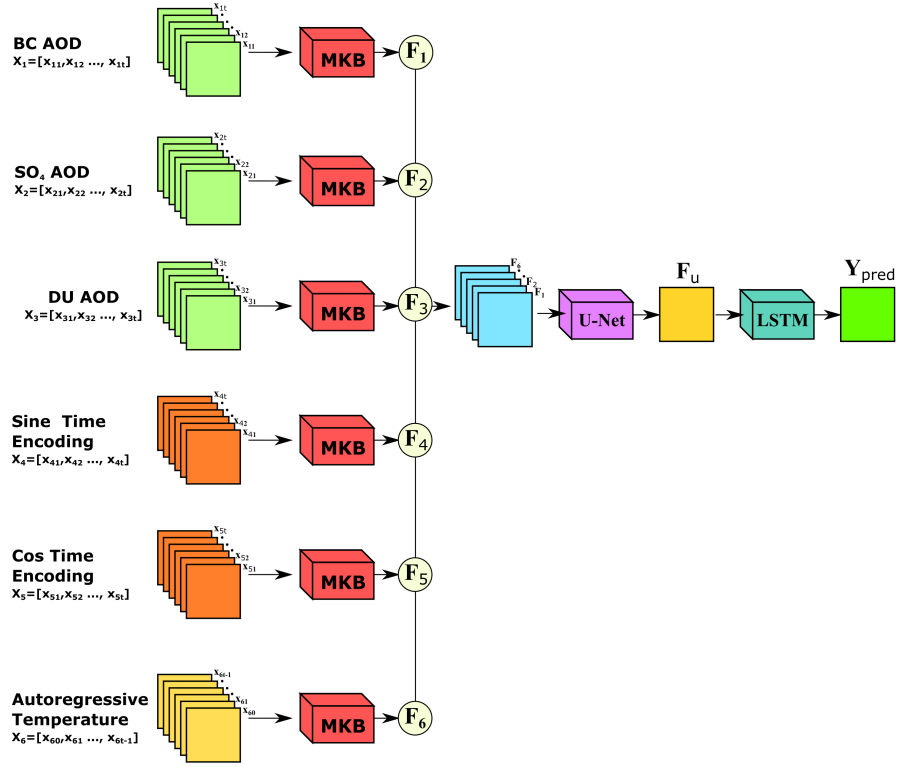


Fig. 1.1: Overview of the proposed Multivariate Spatio-Temporal AI model for Temperature Forecasting from Aerosol Optical Depth

1.6.3.1 Multi-Kernel Block for Feature Extraction

The first component processes each input channel to create feature representations. This component uses depthwise convolution, which applies different filters to different channels. This design preserves the identity of each input variable while extracting spatial patterns.

The depthwise convolution operates on each input channel independently using multiple kernel sizes simultaneously. The architecture processes each channel with several spatial filters of different sizes. A three by three kernel looks at a local neighborhood of nine pixels. A five by five kernel examines a broader region of twenty-five pixels. A seven by seven kernel captures an even wider context of forty-nine pixels. Each kernel size captures different scales of spatial patterns.

Each kernel size produces one output channel per input channel. With three kernel sizes applied to six input channels, the operation generates eighteen feature channels. The first six

channels come from the three by three kernels, the next six from the five by five kernels, and the final six from the seven by seven kernels. All outputs concatenate to form the complete feature representation.

The mathematical operation at each spatial location involves a weighted sum. The weights form a small spatial filter. For a three by three kernel, nine weights define the filter. For a five by five kernel, twenty-five weights define the filter. For a seven by seven kernel, forty-nine weights define the filter. Different sets of weights apply to different input channels, ensuring each aerosol type receives its own specialized processing.

This multi-scale approach allows the model to capture both fine details and broader context simultaneously. Small kernels detect local patterns like edges and textures. Large kernels recognize extended structures and regional relationships. Processing all scales together creates a rich representation that captures features across the full range of spatial extents.

After the convolution, batch normalization standardizes the features. This normalization uses statistics computed across the batch dimension. The transformation includes learnable scale and shift parameters. These parameters give the model flexibility to adjust the feature distributions during training.

A ReLU activation function follows the batch normalization. This function replaces negative values with zero. This introduces nonlinearity, which allows the model to learn complex patterns. The ReLU function is simple and works well in practice.

The multi-kernel block serves two purposes. First, it expands the feature dimension. This expansion gives the model more capacity to represent different aspects of the input. Second, it applies local spatial processing. This processing captures patterns like spatial gradients and local variations.

Also, the multi-kernel block design preserves channel identity while extracting features. This matters because different aerosol types have different physical meanings. Black carbon, sulfate, and dust aerosols affect temperature through different mechanisms.

Processing each channel separately prevents the model from confusing these different aerosol types. The model learns specific relationships between each aerosol type and temperature. This produces more interpretable and physically grounded predictions.

Mathematical Formulation

For input

$$\mathbf{X}_t \in \mathbb{R}^{B \times C \times 19 \times 19}$$

Standard Depthwise Convolution:

$$Z_t = \text{DWConv}_{k \times k}(X_t) \in \mathbb{R}^{B \times (C \cdot m) \times H \times W} \quad (1.6.15)$$

where:

- k : kernel size
- m : output multiplier per channel

Multi-Kernel Mode: Multiple Receptive Fields

In multi-kernel mode, different kernel sizes process each channel independently, creating diverse receptive fields.

Multiple Depthwise Convolutions:

Given kernel sizes $\mathcal{K} = \{k_1, k_2, \dots, k_M\}$ (e.g., $\{3, 5, 7\}$):

$$Z_t^{(i)} = \text{DWConv}_{k_i \times k_i}(X_t) \in \mathbb{R}^{B \times C \times H \times W}, \quad i = 1, \dots, M \quad (1.6.16)$$

Concatenation:

$$Z_t = [Z_t^{(1)}, Z_t^{(2)}, \dots, Z_t^{(M)}] \in \mathbb{R}^{B \times (C \cdot M) \times H \times W} \quad (1.6.17)$$

Batch Normalization:

$$\hat{Z}_t = \text{BN}(Z_t) = \gamma \odot \frac{Z_t - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}} + \beta \quad (1.6.18)$$

where:

- $\mu_{\text{batch}}, \sigma_{\text{batch}}^2$: batch statistics per channel
- γ, β : learnable scale and shift parameters
- $\epsilon = 10^{-5}$: numerical stability constant

ReLU Activation:

$$F_t^{\text{MKB}} = \text{ReLU}(\hat{Z}_t) = \max(0, \hat{Z}_t) \in \mathbb{R}^{B \times C' \times H \times W} \quad (1.6.19)$$

where $C' = C \cdot m$ (standard mode) or $C' = C \cdot M$ (multi-kernel mode).

Therefore, MKB learns:

- **Local spatial patterns**
- **Channel-independent features** preserving variable identity
- **Feature expansion** from C to $C \cdot k$ channels
- **Non-linear transformations** through BN + ReLU

1.6.3.2 U-Net for Multi-Scale Spatial Processing

The U-Net architecture processes features at multiple spatial resolutions. This multi-scale approach captures both fine details and broad patterns. The architecture includes an encoder path, a bottleneck, and a decoder path.

The U-Net also includes squeeze-and-excitation (SE) blocks at each level. These blocks implement channel attention. Channel attention allows the model to emphasize important features and suppress less relevant ones.

The squeeze operation computes global statistics for each feature channel. It applies global average pooling across spatial dimensions. This produces one number per channel that summarizes the spatial information. This operation compresses the spatial dimensions while preserving the channel dimension.

The excitation operation processes these channel statistics through two fully connected layers. The first layer reduces dimensionality by a factor of eight. A ReLU activation introduces nonlinearity. The second layer restores the original channel dimension. A sigmoid activation produces values between zero and one.

These values act as channel-wise attention weights. Each channel multiplies by its corresponding weight. Channels with high weights contribute more to subsequent processing.

Channels with low weights contribute less. The model learns these weights during training based on which channels help predict temperature.

The squeeze-and-excitation mechanism adds minimal parameters. The two fully connected layers contain far fewer parameters than the convolutional layers. The computational cost is also small because the mechanism operates on compressed channel statistics rather than full spatial features.

The U-Net serves the purpose of capturing both scales through its encoder-decoder structure i.e. Local hotspots span a few grid cells & regional patterns span dozens of grid cells.

As the temperature patterns appear at multiple spatial scales, U-Net's encoder-decoder structure serves to capture both scales. i.e. Local hotspots span a few grid cells & regional patterns span dozens of grid cells.

The encoder extracts increasingly abstract features. Early layers capture fine details. Later layers capture broad patterns. The decoder combines these scales through skip connections. This produces predictions that respect both local and regional structure.

Alternative single-scale architectures would miss either fine details or broad patterns. The multi-scale approach captures the full range of spatial phenomena.

Mathematical Formulation

The SE block performs channel-wise attention:

Squeeze Operation (Global Average Pooling)

$$\mathbf{z}_c = \text{GAP}(\mathbf{U}_c) = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W u_c(i, j) \quad (1.6.20)$$

Excitation Operation (Gating)

$$\mathbf{s} = \sigma(\mathbf{W}_2 \cdot \text{ReLU}(\mathbf{W}_1 \cdot \mathbf{z})) \quad (1.6.21)$$

where:

- $\mathbf{W}_1 \in \mathbb{R}^{\frac{C}{r} \times C}$: dimensionality reduction (reduction ratio $r = 8$)
- $\mathbf{W}_2 \in \mathbb{R}^{C \times \frac{C}{r}}$: dimensionality restoration
- σ : sigmoid activation producing channel weights

Feature Recalibration

$$\mathbf{U}_c^{\text{SE}} = s_c \cdot \mathbf{U}_c \quad (1.6.22)$$

Therefore, SE learns:

- **Channel importance:** Which feature maps are most relevant
- **Adaptive recalibration:** Context-dependent feature weighting
- **Global context:** Via global average pooling

Double Convolution with SE

$$\text{DoubleConvSE}(\mathbf{X}) = \text{SE}(\text{ReLU}(\text{BN}(\text{Conv}_2(\text{ReLU}(\text{BN}(\text{Conv}_1(\mathbf{X}))))) \quad (1.6.23)$$

The Components of U-Net are discussed in detail below:

1. **Encoder Path:** The encoder path progressively reduces spatial resolution while increasing feature depth. This path consists of four levels. Each level applies two convolutional blocks followed by spatial downsampling.

The first level operates at the full 19×19 resolution. The convolutional blocks use three by three kernels with padding. This maintains the spatial size within each level. Batch normalization and ReLU activation follow each convolution. After processing at this level, bilinear interpolation reduces the spatial size to 13×13 .

The second level processes features at this reduced resolution. The number of feature channels doubles compared to the previous level. This increase in depth allows the model to represent more complex patterns. After processing, interpolation reduces the size to 8×8 .

The third and fourth levels continue this pattern. The spatial size decreases to 3×3 , then to 1×1 . The feature depth continues to increase. This progression creates a pyramid of features at different scales.

The encoder design follows a standard principle in computer vision. As spatial resolution decreases, the model can afford to use more feature channels. The total computational cost remains roughly balanced across levels. Early levels process more spatial locations with fewer features. Later levels process fewer locations with more features.

Mathematical Formulation

Encoder Level 0 (19×19)

$$\mathbf{e}_0 = \text{DoubleConvSE}(\mathbf{F}_t^{\text{MKB}}) \in \mathbb{R}^{B \times 16 \times 19 \times 19} \quad (1.6.24)$$

Encoder Level 1 (13×13)

$$\mathbf{e}_1 = \text{DoubleConvSE}(\text{Resize}(\mathbf{e}_0, 13)) \in \mathbb{R}^{B \times 32 \times 13 \times 13} \quad (1.6.25)$$

Encoder Level 2 (8×8)

$$\mathbf{e}_2 = \text{DoubleConvSE}(\text{Resize}(\mathbf{e}_1, 8)) \in \mathbb{R}^{B \times 64 \times 8 \times 8} \quad (1.6.26)$$

Encoder Level 3 (3×3)

$$\mathbf{e}_3 = \text{DoubleConvSE}(\text{Resize}(\mathbf{e}_2, 3)) \in \mathbb{R}^{B \times 128 \times 3 \times 3} \quad (1.6.27)$$

2. **Bottleneck:** The bottleneck operates at one by one spatial resolution. At this point, each feature vector represents the entire spatial grid. The bottleneck contains the deepest feature representation with the most channels.

Two design options exist for the bottleneck. The standard option uses two three by three convolutions. Despite the 1×1 spatial size, these convolutions still transform the feature channels. The alternative option uses 1×1 convolutions. These convolutions mix channel information without spatial processing. This reduces parameters while maintaining representation capacity.

Mathematical Formulation

Standard Bottleneck:

$$\mathbf{b} = \text{DoubleConvSE}(\text{Resize}(\mathbf{e}_3, 1)) \in \mathbb{R}^{B \times 256 \times 1 \times 1} \quad (1.6.28)$$

1×1 Bottleneck:

$$\mathbf{b} = \text{ReLU}(\text{BN}(\text{Conv}_{1 \times 1}^{(2)}(\text{ReLU}(\text{BN}(\text{Conv}_{1 \times 1}^{(1)}(\text{Resize}(\mathbf{e}_3, 1))))))) \quad (1.6.29)$$

3. **Decoder Path:** The decoder path reverses the encoder process. It progressively increases spatial resolution while decreasing feature depth. The decoder mirrors the encoder structure but adds skip connections.

Skip connections transfer features from the encoder to the decoder. At each level, the decoder receives features from the corresponding encoder level. This transfer happens through concatenation. The decoder concatenates upsampled features from the previous level with encoder features from the current level.

These skip connections serve an important purpose. The encoder path compresses spatial information into deep features. Some spatial details may be lost during this compression. The skip connections provide a direct path for these details to reach the decoder. This helps the model reconstruct sharp spatial patterns.

The decoder applies the same processing as the encoder. Each level uses two convolutional blocks with batch normalization and ReLU. Between levels, bilinear interpolation increases the spatial size. The upsampling follows a fixed schedule that matches the encoder downsampling.

The final decoder level outputs features at the original 19 by 19 resolution. A one by one convolution reduces these features to a single channel. This channel represents the spatial temperature prediction for one time step.

Mathematical Formulation

Decoder Level 3 (3×3)

$$\mathbf{d}_3 = \text{DoubleConvSE}([\text{Resize}(\text{Conv}_{1 \times 1}(\mathbf{b}), 3), \mathbf{e}_3]) \in \mathbb{R}^{B \times 64 \times 3 \times 3} \quad (1.6.30)$$

Decoder Level 2 (8×8)

$$\mathbf{d}_2 = \text{DoubleConvSE}([\text{Resize}(\text{Conv}_{1 \times 1}(\mathbf{d}_3), 8), \mathbf{e}_2]) \in \mathbb{R}^{B \times 32 \times 8 \times 8} \quad (1.6.31)$$

Decoder Level 1 (13×13)

$$\mathbf{d}_1 = \text{DoubleConvSE}([\text{Resize}(\text{Conv}_{1 \times 1}(\mathbf{d}_2), 13), \mathbf{e}_1]) \in \mathbb{R}^{B \times 16 \times 13 \times 13} \quad (1.6.32)$$

Decoder Level 0 (19×19)

$$\mathbf{d}_0 = \text{DoubleConvSE}([\text{Resize}(\text{Conv}_{1 \times 1}(\mathbf{d}_1), 19), \mathbf{e}_0]) \in \mathbb{R}^{B \times 16 \times 19 \times 19} \quad (1.6.33)$$

U-Net Output

$$\mathbf{S}_t = \text{Conv}_{1 \times 1}(\mathbf{d}_0) \in \mathbb{R}^{B \times 1 \times 19 \times 19} \quad (1.6.34)$$

1.6.3.3 Long Short-Term Memory (LSTM) for Temporal Sequence Processing

The U-Net processes each time step independently. It produces a spatial feature map for each input frame. These feature maps need temporal processing to generate predictions that respect temporal dynamics. A long short-term memory network handles this temporal processing. The architecture of LSTM is shown Figure 1.2

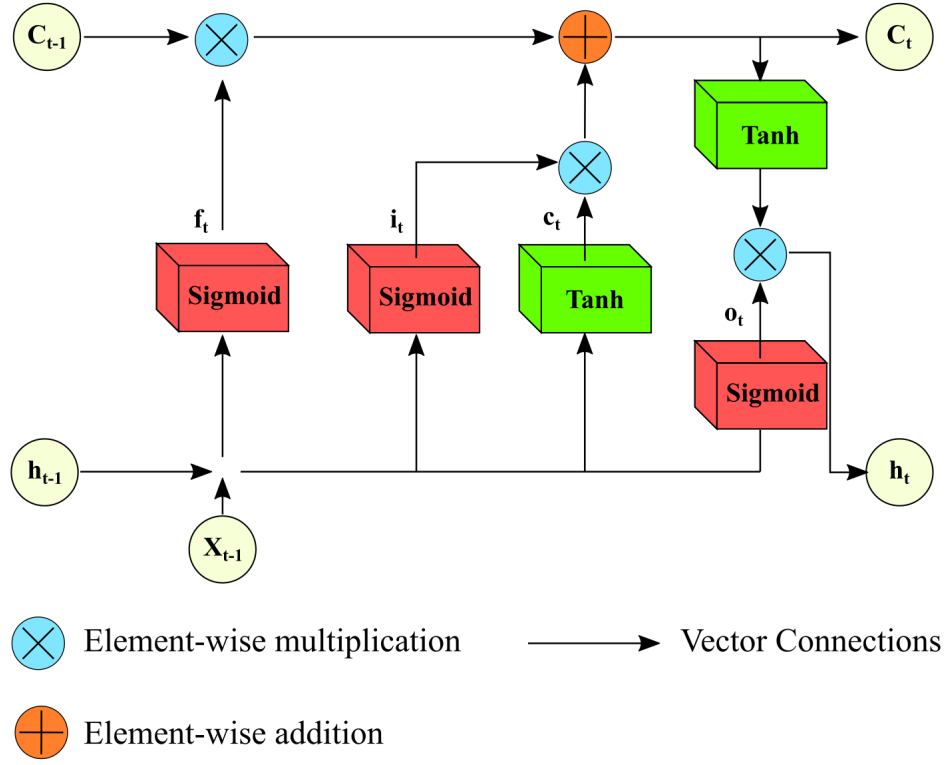


Fig. 1.2: Schematic illustration of the Long Short-Term Memory (LSTM) block used for temporal modeling in the proposed framework.

Feature Sequence Preperation

Before temporal processing, the model flattens each spatial feature map. A 19×19 spatial map with one channel becomes a 361-dimensional vector. The model creates one such vector for each time step in the sequence. These vectors stack to form a sequence with shape batch by time by 361.

This flattening removes explicit spatial structure. The LSTM receives features as vectors rather than spatial grids. This simplifies the LSTM processing but discards spatial adjacency information. The U-Net already processed spatial relationships, so this information exists within the feature vectors.

Mathematical Formulation

Flatten spatial features from each timestep:

$$s_t = \text{Flatten}(S_t) \in \mathbb{R}^{361}, \quad t = 1, 2, \dots, T \quad (1.6.35)$$

where $361 = 19 \times 19$.

Stack into sequence:

$$\mathcal{S} = [s_1, s_2, \dots, s_T] \in \mathbb{R}^{B \times T \times 361} \quad (1.6.36)$$

LSTM Architecture

The model uses a two-layer LSTM. Each layer maintains hidden states and cell states across time steps. The hidden state captures short-term patterns. The cell state captures long-term dependencies.

The first LSTM layer processes the input sequence. At each time step, it receives a 361-dimensional feature vector. It combines this input with the hidden state from the previous time step. The combination passes through four gates: forget, input, candidate, and output.

The forget gate decides which information to remove from the cell state. It uses a sigmoid activation that produces values between zero and one. Values near zero indicate information to forget. Values near one indicate information to keep. This selective memory allows the LSTM to discard irrelevant historical information.

The input gate decides which new information to add to the cell state. It also uses a sigmoid activation. The candidate gate computes potential new information using a tanh activation. The element-wise product of the input gate and candidate gate determines what actually enters the cell state.

The cell state update combines these operations. It multiplies the old cell state by the forget gate output. It adds the product of the input gate and candidate gate. This creates a weighted combination of old information and new information.

The output gate controls which parts of the cell state affect the hidden state. It applies a sigmoid activation to determine the gating values. The cell state passes through a tanh activation and multiplies by these gating values. The result becomes the new hidden state.

The second LSTM layer receives the hidden states from the first layer. Before this transfer, dropout randomly sets some values to zero during training. The dropout probability is 0.4, meaning forty percent of values become zero. This regularization prevents overfitting by forcing the model to distribute information across many units.

The second LSTM layer follows the same gate structure as the first layer. It produces its own hidden states and cell states. These states capture higher-level temporal patterns built

on top of the first layer's representations.

Mathematical Formulation

LSTM Architecture Configuration:

- Number of layers: 2
- Hidden dimension: $d_h = 96$
- Dropout: $p = 0.4$ (between layers)

Layer 1 LSTM Dynamics

For each timestep t :

Forget Gate:

$$f_t^{(1)} = \sigma(W_f^{(1)} \cdot [h_{t-1}^{(1)}, s_t] + b_f^{(1)}) \quad (1.6.37)$$

Input Gate:

$$i_t^{(1)} = \sigma(W_i^{(1)} \cdot [h_{t-1}^{(1)}, s_t] + b_i^{(1)}) \quad (1.6.38)$$

Candidate Cell State:

$$\tilde{c}_t^{(1)} = \tanh(W_c^{(1)} \cdot [h_{t-1}^{(1)}, s_t] + b_c^{(1)}) \quad (1.6.39)$$

Cell State Update:

$$c_t^{(1)} = f_t^{(1)} \odot c_{t-1}^{(1)} + i_t^{(1)} \odot \tilde{c}_t^{(1)} \quad (1.6.40)$$

Output Gate:

$$o_t^{(1)} = \sigma(W_o^{(1)} \cdot [h_{t-1}^{(1)}, s_t] + b_o^{(1)}) \quad (1.6.41)$$

Hidden State:

$$h_t^{(1)} = o_t^{(1)} \odot \tanh(c_t^{(1)}) \quad (1.6.42)$$

Dropout Between Layers

$$\tilde{h}_t^{(1)} = \text{Dropout}(h_t^{(1)}, p = 0.4) \quad (1.6.43)$$

During training, each element is randomly set to 0 with probability 0.4, and scaled by $1/(1 - p)$.

Layer 2 LSTM Dynamics

Similar dynamics with $\tilde{h}_t^{(1)}$ as input:

$$h_t^{(2)}, c_t^{(2)} = \text{LSTM}^{(2)}(\tilde{h}_t^{(1)}, h_{t-1}^{(2)}, c_{t-1}^{(2)}) \quad (1.6.44)$$

Complete LSTM Output

All hidden states from Layer 2:

$$\mathcal{H} = [h_1^{(2)}, h_2^{(2)}, \dots, h_T^{(2)}] \in \mathbb{R}^{B \times T \times d_h} \quad (1.6.45)$$

LSTMs solve the problem of long-term dependencies in temperature evolution. For instance, aerosol conditions from several time steps ago can still affect current temperatures, but simple recurrent neural networks struggle to capture these patterns due to vanishing gradients. LSTMs overcome this problem through their cell state mechanism, which preserves information over many time steps. Their gates control what information enters, exits, and persists in the cell state, allowing the model to learn which historical patterns truly matter for the prediction.

Alternatives like gated recurrent units could also work. LSTMs were chosen due to their established track record and widespread use.

1.6.3.4 Output Generation

1.6.4 Physics Informed Loss Function

The loss function guides model training. A well-designed loss function encourages the model to learn physically plausible patterns. This proposed framework uses a multi-component loss function that combines four distinct objectives.

Mathematical Formulation

The loss combines four complementary objectives:

$$\mathcal{L}_{\text{total}} = \lambda_1 \mathcal{L}_{\text{Huber}} + \lambda_2 \mathcal{L}_{\text{temp}} + \lambda_3 \mathcal{L}_{\text{grad}} + \lambda_4 \mathcal{L}_{\text{SSIM}} \quad (1.6.46)$$

The components of loss function are discussed below:

1.6.4.1 Huber Loss

The first component measures prediction accuracy at each spatial location. It compares predicted temperature values to actual temperature values. The comparison uses Huber loss rather than mean squared error.

Huber loss combines properties of squared error and absolute error. For small errors, it behaves like squared error. This provides smooth gradients that help optimization. For large errors, it behaves like absolute error. This reduces the impact of outliers.

The transition between these behaviors occurs at a threshold parameter δ . When the absolute difference between prediction and target is less than δ , the loss equals one half of the squared difference. When the absolute difference exceeds δ , the loss equals δ times the absolute difference minus one half of δ^2 .

The research uses $\delta = 0.5$. This relatively small threshold makes the loss robust to outliers. Temperature predictions occasionally produce values far from the target due to challenging atmospheric conditions. Huber loss prevents these outliers from dominating the gradient signal.

The basic Huber loss averages over all examples, time steps, and spatial locations. This gives each prediction equal weight. An optional enhancement applies temporal weighting. This weighting emphasizes recent time steps over early time steps in the sequence.

The temporal weights follow an exponential schedule. Early time steps receive lower weights. Later time steps receive higher weights. The weights sum to one across the sequence. A parameter γ controls the steepness of the exponential. Higher γ values create stronger emphasis on recent predictions.

This temporal weighting reflects forecast priorities. In many applications, accurate near-term predictions matter more than accurate long-term predictions. The model learns to prioritize recent time steps when this weighting applies.

Mathematical Formulation

$$\mathcal{L}_{\text{Huber}}(\hat{Y}, Y) = \frac{1}{B \cdot T \cdot 361} \sum_{b,t,i,j} L_{\delta}(\hat{y}_{b,t,i,j}, y_{b,t,i,j}) \quad (1.6.47)$$

where:

$$L_{\delta}(a, b) = \begin{cases} \frac{1}{2}(a - b)^2 & \text{if } |a - b| \leq \delta \\ \delta \left(|a - b| - \frac{1}{2}\delta \right) & \text{otherwise} \end{cases} \quad (1.6.48)$$

Default: $\delta = 0.5$

1.6.4.2 Temporal Consistency Loss

Temperature changes smoothly over time in real atmospheric conditions. Large sudden jumps in temperature are rare. The temporal consistency loss encourages the model to respect this physical constraint.

The loss compares temperature changes in predictions to temperature changes in targets. For each time step after the first, it computes the difference between temperature at time t and temperature at time t minus one. This difference represents the temperature change or temporal gradient.

The loss measures the mean squared error between predicted changes and actual changes. If the model predicts smooth evolution and the actual evolution is smooth, this loss is low. If the model predicts jumps while the actual evolution is smooth, this loss is high.

This constraint embodies the concept of atmospheric inertia. Atmospheric temperature cannot change instantaneously. Physical processes like radiation and convection operate on finite time scales. The model must learn these time scales implicitly through the temporal consistency loss.

The weight on this loss component is 0.7 by default. This substantial weight reflects the importance of temporal smoothness. Without this constraint, the model might learn to make independent predictions at each time step. These predictions might be individually accurate but temporally inconsistent.

Mathematical Formulation

Enforces smooth temporal evolution (atmospheric inertia):

$$\mathcal{L}_{\text{temp}} = \frac{1}{B \cdot (T - 1)} \sum_{b=1}^B \sum_{t=2}^T \|\Delta \hat{Y}_{b,t} - \Delta Y_{b,t}\|_2^2 \quad (1.6.49)$$

where temporal differences are:

$$\Delta \hat{Y}_{b,t} = \hat{Y}_{b,t} - \hat{Y}_{b,t-1}, \quad \Delta Y_{b,t} = Y_{b,t} - Y_{b,t-1} \quad (1.6.50)$$

Physical interpretation:

$$\left. \frac{\partial T}{\partial t} \right|_{\text{pred}} \approx \left. \frac{\partial T}{\partial t} \right|_{\text{true}} \quad (1.6.51)$$

Temperature change rates in predictions should match reality, embodying the principle that atmospheric temperature evolves gradually due to thermal inertia.

1.6.4.3 Spatial Gradient Loss

Temperature varies across space in structured ways. Fronts create sharp gradients. Stable conditions create smooth fields. The spatial gradient loss helps the model reproduce these spatial patterns.

The loss examines gradients in two directions: horizontal and vertical. The horizontal gradient at each location equals the difference between temperature at the current location and temperature at the location to the right. The vertical gradient equals the difference between current temperature and temperature at the location below.

The loss computes these gradients for both predictions and targets. It then measures the mean absolute error between predicted gradients and actual gradients. This comparison happens separately for horizontal and vertical gradients. The sum of these errors becomes the spatial gradient loss.

This formulation preserves edge sharpness. Locations with strong gradients in the target will have strong gradients in the prediction. Locations with weak gradients in the target will have weak gradients in the prediction. The model learns to reproduce spatial structure rather than just spatial averages.

The gradient loss uses absolute error rather than squared error. This choice makes the

loss less sensitive to the magnitude of gradients. It focuses on matching the pattern of gradients across space. This works well because gradient magnitudes can vary substantially across different weather conditions.

The weight on spatial gradient loss is 0.3 by default. This moderate weight balances structure preservation with magnitude accuracy. Too much weight on gradients could cause the model to emphasize edges while sacrificing overall accuracy. Too little weight could produce smooth predictions that miss important spatial details.

Mathematical Formulation

Preserves spatial structure and sharpness:

$$\mathcal{L}_{\text{grad}} = \mathcal{L}_{\nabla_x} + \mathcal{L}_{\nabla_y} \quad (1.6.52)$$

Horizontal Gradient Loss

$$\mathcal{L}_{\nabla_x} = \frac{1}{N} \sum \left\| \left| \frac{\partial \hat{Y}}{\partial x} \right| - \left| \frac{\partial Y}{\partial x} \right| \right\| \quad (1.6.53)$$

where:

$$\left. \frac{\partial \hat{Y}}{\partial x} \right|_{i,j} \approx \hat{Y}_{i,j+1} - \hat{Y}_{i,j} \quad (1.6.54)$$

Vertical Gradient Loss

$$\mathcal{L}_{\nabla_y} = \frac{1}{N} \sum \left\| \left| \frac{\partial \hat{Y}}{\partial y} \right| - \left| \frac{\partial Y}{\partial y} \right| \right\| \quad (1.6.55)$$

where:

$$\left. \frac{\partial \hat{Y}}{\partial y} \right|_{i,j} \approx \hat{Y}_{i+1,j} - \hat{Y}_{i,j} \quad (1.6.56)$$

Physical interpretation: Preserves temperature fronts, and spatial discontinuities that characterize real atmospheric fields.

1.6.4.4 Structural Similarity (SSIM)

The structural similarity index measures how well two images match in terms of structure rather than raw values. This metric originally came from image quality assessment. It

applies well to spatial prediction problems.

The metric compares three aspects: luminance, contrast, and structure. Luminance refers to average intensity. Contrast refers to intensity variation. Structure refers to patterns after removing luminance and contrast effects.

The computation uses local windows rather than global statistics. A Gaussian window of size seven by seven slides across the spatial grid. At each position, the window weights define which pixels contribute to the local statistics. Pixels near the window center receive higher weights. Pixels near the window edge receive lower weights.

For each window position, the metric computes local means, variances, and covariance. The luminance comparison uses the local means. The contrast comparison uses the local standard deviations. The structure comparison uses the local covariance normalized by the standard deviations.

These three comparisons combine into a single similarity score between zero and one. A score of one indicates perfect similarity. A score of zero indicates no similarity. The loss equals one minus this similarity score. This converts the similarity measure into a loss function that decreases as predictions improve.

The structural similarity loss can apply to individual time steps or to all time steps. Applying it to all time steps increases computational cost but improves temporal consistency. The default configuration applies it to all time steps.

The weight on structural similarity loss is 0.1 by default. This small weight provides a subtle signal that complements the other loss components. The metric is scale-invariant, which means it focuses on patterns rather than absolute magnitudes. This makes it a useful supplement to the Huber loss, which focuses on magnitudes.

Mathematical Formulation

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)} \quad (1.6.57)$$

where:

- μ_x, μ_y : local means (computed via Gaussian filtering)
- σ_x^2, σ_y^2 : local variances

- σ_{xy} : local covariance
- $c_1 = (0.01 \cdot \text{range})^2$, $c_2 = (0.03 \cdot \text{range})^2$: stability constants

SSIM Loss

$$\mathcal{L}_{\text{SSIM}} = 1 - \text{SSIM}(\hat{Y}, Y) \quad (1.6.58)$$

SIM captures:

- Luminance: $l(x, y) = \frac{2\mu_x\mu_y+c_1}{\mu_x^2+\mu_y^2+c_1}$
- Contrast: $c(x, y) = \frac{2\sigma_x\sigma_y+c_2}{\sigma_x^2+\sigma_y^2+c_2}$
- Structure: $s(x, y) = \frac{\sigma_{xy}+c_2/2}{\sigma_x\sigma_y+c_2/2}$

Finally, the total loss sums the four components with their respective weights. The weights are hyperparameters that control the relative importance of each objective.

These weights were chosen through experimentation. The Huber loss receives the highest weight because magnitude accuracy is the primary goal. The temporal consistency loss receives the second highest weight because smooth temporal evolution is crucial for credible forecasts. The spatial gradient and structural similarity losses receive lower weights because they refine the predictions rather than define their basic accuracy.

The loss function is differentiable. Each component produces gradients with respect to model parameters. The optimizer uses these gradients to update parameters during training. The multi-component design provides rich gradient signals that guide learning in multiple directions simultaneously.

1.6.5 Evaluation Metrics

To quantitatively evaluate the performance of the proposed Atten-U-Net-LSTM model and to facilitate comparison with baseline methods, four standard evaluation metrics are employed: **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, and the **Structural Similarity Index Measure (SSIM)**. These metrics collectively assess prediction accuracy, error magnitude, and structural similarity between predicted and observed PM_{2.5} fields.

1.6.5.1 Mean Squared Error (MSE)

MSE penalizes larger errors more strongly due to the squaring operation and is given by

$$\text{MSE}(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2. \quad (1.6.59)$$

1.6.5.2 Root Mean Squared Error (RMSE)

RMSE is the square root of MSE and has the same physical units as the target variable:

$$\text{RMSE}(\mathbf{y}, \hat{\mathbf{y}}) = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (1.6.60)$$

RMSE is particularly sensitive to large deviations and is widely used in air quality modeling studies.

1.6.5.3 Structural Similarity Index Measure (SSIM)

To evaluate the spatial structural consistency between predicted and observed $\text{PM}_{2.5}$ concentration maps, SSIM is employed. It is defined as

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}, \quad (1.6.61)$$

where μ_x and μ_y are the mean intensities of the predicted and observed fields, σ_x^2 and σ_y^2 are their variances, and σ_{xy} is the covariance between them. The constants c_1 and c_2 are included to stabilize the division in the presence of small denominators.

SSIM captures structural similarity in terms of luminance, contrast, and spatial pattern, making it particularly suitable for evaluating gridded $\text{PM}_{2.5}$ concentration maps beyond pointwise error statistics.

1.6.6 Comparison with Baseline Models

1.6.6.1 Convolutional LSTM for Spatio-Temporal Sequence Learning

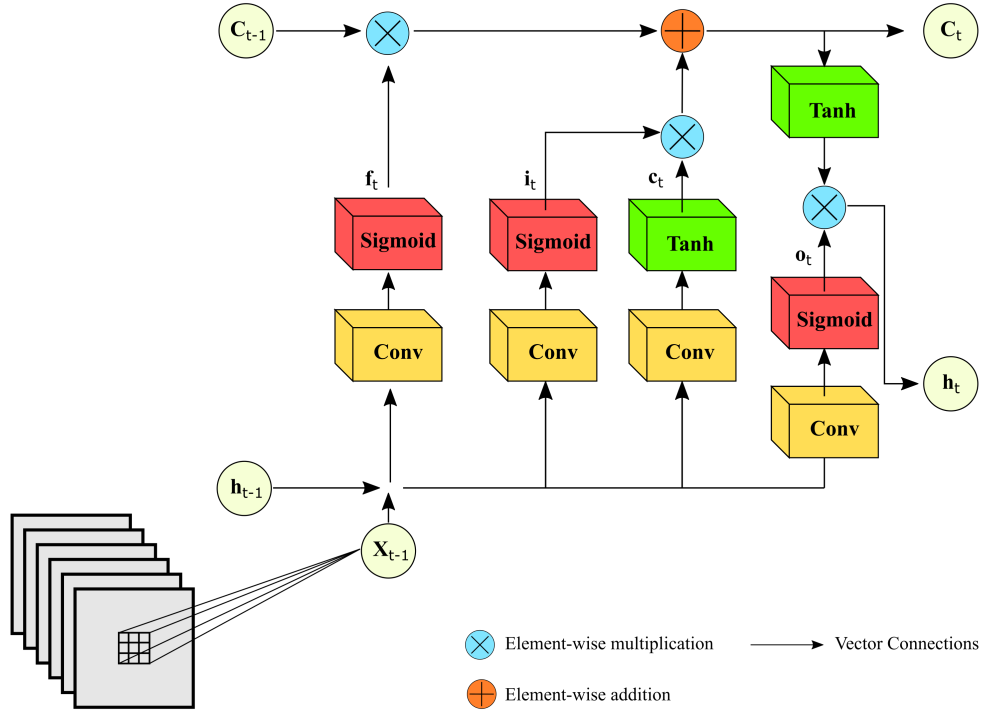


Fig. 1.3: Architecture of Convolutional LSTM.

Convolutional Long Short-Term Memory (ConvLSTM) network proposed by Shi et al. [Shi2015], is based on the standard LSTM architecture with a convolutional operator in both state-to-state and input-to-state transitions to be able to capture both spatio-temporal data. ConvLSTM is tailored to two-dimensional structured inputs, e.g., an image or a grid of geophysical measurements, as seen in Figure ?? above.

A ConvLSTM uses an input at time t as a multi-channel spatial tensor instead of a one-dimensional vector, and the hidden and cell states are represented using their spatial time structure. Thus, the gating functions of the LSTM are represented by the input, forget and output gates, which are computed through convolution in the spatial grid. This enables ConvLSTM to collectively learn both spatial relations and temporal relations under one system.

Formally, let $\mathbf{X}_t$ denote the input field (e.g., precipitation or meteorological variables) at time t , represented as a two-dimensional grid. The ConvLSTM updates are given by

$$\mathbf{i}_t = \sigma(\mathbf{W}_{xi} * \mathbf{X}_t + \mathbf{W}_{hi} * \mathbf{H}_{t-1} + \mathbf{W}_{ci} \circ \mathbf{C}_{t-1} + \mathbf{b}_i), \quad (1.6.62)$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_{xf} * \mathbf{X}_t + \mathbf{W}_{hf} * \mathbf{H}_{t-1} + \mathbf{W}_{cf} \circ \mathbf{C}_{t-1} + \mathbf{b}_f), \quad (1.6.63)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_{xo} * \mathbf{X}_t + \mathbf{W}_{ho} * \mathbf{H}_{t-1} + \mathbf{W}_{co} \circ \mathbf{C}_t + \mathbf{b}_o), \quad (1.6.64)$$

$$\mathbf{C}_t = \mathbf{f}_t \circ \mathbf{C}_{t-1} + \mathbf{i}_t \circ \tanh(\mathbf{W}_{xc} * \mathbf{X}_t + \mathbf{W}_{hc} * \mathbf{H}_{t-1} + \mathbf{b}_c), \quad (1.6.65)$$

$$\mathbf{H}_t = \mathbf{o}_t \circ \tanh(\mathbf{C}_t), \quad (1.6.66)$$

where a convolution operator is denoted by a $*$, the element-wise (Hadamard) multiplication operation by a symbol $\circ$, and the sigmoid activation function is denoted by $\sigma(\cdot)$. The hidden state $\mathbf{H}_t$ and cell state $\mathbf{C}_t$ are both spatial feature maps that maintain two-dimensional structure across time.

ConvLSTM can learn temporal interactions that are spatially local as a result of convolutional kernel (i.e., $\mathbf{W}_{xi}$ and $\mathbf{W}_{hi}$, allowing neighbouring grid cells to affect the future state of each other. It is a preservation of coherent propagation of physical processes like movement, growth and decay of precipitation or pollution patterns across space and time.

1.6.6.1.1 ConvLSTM versus LSTM The main difference between ConvLstm and Lstm is the inputs and states representation. The classic LSTMs use one-dimensional vectors and thus ignore explicit spatial relations. ConvLSTM, conversely, uses three-dimensional tensors (rows, columns, channels) and each gate calculation takes the form of a convolution through the spatial domain instead of a matrix multiplication. Thus, local neighbourhood in the previous hidden state affects the hidden state of a certain position, (i, j) depending on the size of the convolution kernel. For example, a 3 x 3 kernel allows the grid cell to be informed with the current cell together with the eight neighbouring cells of the prior time step.

This is a spatially conscious time-dependent modelling that is specifically significant to the atmospheric processes involving precipitation, where the weather features are highly spatially continuous and localised in movement. ConvLSTM has been broadly demonstrated

to be better than either CNN or LSTM models in terms of spatio-temporal prediction. Its applicability in geophysical modelling as its generality and strength in short-term precipitation forecasting have been established in the past studies [Kim2017], in closely related problems, including soil moisture modelling and prediction of wind speed, [OtherStudies]

1.6.6.2 Dual-stage attention-based Conv-LSTM

Dual-Stage Attention-based Convolutional Long Short-Term Memory (DA-Conv-LSTM) network proposed by Xiao et al. [Xiao2021], extends the standard LSTM architecture by incorporating a convolutional layer for spatial feature extraction and a two-stage attention mechanism for selective temporal weighting. The model is designed for multivariate time series (MTS) prediction, where the predicted value depends not only on the past values of the target series but also on a set of exogenous input series.

DA-Conv-LSTM treats each time step of the MTS as a two-dimensional spatial frame, analogous to a single frame in a video sequence. A convolutional layer first operates on this spatial structure to extract inter-variable relationships at each time step. The extracted spatial features then pass through two sequential attention-LSTM stages: the first stage selects relevant exogenous series by assigning adaptive weights to the input, and the second stage selects relevant temporal context by re-weighting the encoder hidden states before decoding the final prediction.

Mathematical Formulation

Given a dataset with n exogenous series and total time length t , the input matrix is defined as:

$$\mathbf{X}_{data} = (\mathbf{x}^1, \mathbf{x}^2, \dots, \mathbf{x}^n) \in \mathbb{R}^{n \times t} \quad (1.6.67)$$

where $\mathbf{x}^k = (x_1^k, x_2^k, \dots, x_t^k)^T \in \mathbb{R}^t$ denotes the k -th exogenous series, and the target series is $\mathbf{y}_{data} = (y_1, y_2, \dots, y_t)^T \in \mathbb{R}^t$. Given a window of size w , the model learns a nonlinear mapping $F(\cdot)$ to predict the next target value:

$$\hat{y}_{w+1} = F(y_1, y_2, \dots, y_w, \mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_w) \quad (1.6.68)$$

Preprocessing

Each time step $\mathbf{x}_l = (x_l^1, x_l^2, \dots, x_l^n) \in \mathbb{R}^n$ is reshaped into a 2D grid of i rows and j columns satisfying:

$$\begin{cases} i = j \\ i \times j = n \end{cases} \quad (1.6.69)$$

When n does not satisfy this condition, zero-padding fills the remaining cells to produce a regular grid. The full window of w time steps is then stacked as a sequence of frames, forming a video-like 3D tensor fed into the convolutional layer.

Spatial Feature Extraction via Convolutional Layer

The convolutional layer operates on the reshaped MTS frames to extract spatial correlation, that is, the relationships between different variables at the same point in time. The output at time step i is:

$$\bar{x}_i = g \left(\sum_{m \in M_i} \mathbf{x}_i * \mathbf{k}_{mi} + \mathbf{b}_{mi} \right), \quad 1 \leq i \leq w \quad (1.6.70)$$

where $g(\cdot)$ is a nonlinear activation function, $*$ denotes the convolution operator, $\mathbf{k}_{mi}$ and $\mathbf{b}_{mi}$ are the learnable kernel weights and biases, and M_i is the receptive field of the kernel. Through the convolutional layer, the model captures complex dependencies among nearby variables in the spatial grid.

First Attention Stage

The convolved output $\bar{\mathbf{x}} = (\bar{x}_1, \bar{x}_2, \dots, \bar{x}_w)$ enters the first attention stage, which computes a scalar weight α_i for each time step by attending to the current LSTM hidden state $\mathbf{h}_w$ and cell state $\mathbf{s}_w$:

$$e_i = \mathbf{v}_e^T \tanh(\mathbf{W}_e[\mathbf{h}_w; \mathbf{s}_w] + \mathbf{U}_e \bar{x}_i), \quad (1.6.71)$$

$$\alpha_i = \frac{\exp(e_i)}{\sum_{i=1}^w \exp(e_i)}, \quad 1 \leq i \leq w \quad (1.6.72)$$

where $\mathbf{v}_e$, $\mathbf{W}_e$, and $\mathbf{U}_e$ are learnable parameters, and $[\mathbf{h}_w; \mathbf{s}_w]$ denotes the concatenation of the hidden state and cell state. The attention weights scale the convolved input to produce the encoder LSTM input:

$$\tilde{\mathbf{x}} = (\alpha_1 \bar{x}_1, \alpha_2 \bar{x}_2, \dots, \alpha_w \bar{x}_w) \quad (1.6.73)$$

Encoder LSTM

The encoder LSTM processes the attention-weighted input $\tilde{\mathbf{x}}$ through its gate structure, where $\circ$ denotes element-wise (Hadamard) multiplication and $\sigma(\cdot)$ denotes the sigmoid activation function:

$$\Gamma^f = \sigma(\mathbf{W}_f [\mathbf{h}_{w-1}; \tilde{\mathbf{x}}] + \mathbf{b}_f), \quad (1.6.74)$$

$$\Gamma^u = \sigma(\mathbf{W}_u [\mathbf{h}_{w-1}; \tilde{\mathbf{x}}] + \mathbf{b}_u), \quad (1.6.75)$$

$$\bar{\mathbf{c}}_w = \tanh(\mathbf{W}_c [\mathbf{h}_{w-1}; \tilde{\mathbf{x}}] + \mathbf{b}_c), \quad (1.6.76)$$

$$\mathbf{s}_w = \Gamma^u \circ \bar{\mathbf{c}}_w + \Gamma^f \circ \mathbf{s}_{w-1}, \quad (1.6.77)$$

$$\Gamma^o = \sigma(\mathbf{W}_o [\mathbf{h}_{w-1}; \tilde{\mathbf{x}}] + \mathbf{b}_o), \quad (1.6.78)$$

$$\mathbf{h}_w = \Gamma^o \circ \tanh(\mathbf{s}_w) \quad (1.6.79)$$

where $\mathbf{W}_f$, $\mathbf{W}_u$, $\mathbf{W}_c$, $\mathbf{W}_o$ are gate weight matrices with corresponding bias terms $\mathbf{b}_f$, $\mathbf{b}_u$, $\mathbf{b}_c$, $\mathbf{b}_o$. The forget gate Γ^f controls what information to discard from the previous cell state, the input gate Γ^u controls what new information to store, and the output gate Γ^o controls what the hidden state exposes to the next stage. The sigmoid function is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (1.6.80)$$

Second Attention Stage

The second attention stage re-weights the encoder hidden states $\mathbf{h}_w = (h_1, h_2, \dots, h_w)$ using the decoder LSTM hidden state $\mathbf{h}'_w$ and cell state $\mathbf{s}'_w$:

$$\boldsymbol{\beta} = f_1(\mathbf{h}'_w, \mathbf{s}'_w, \mathbf{h}_w), \quad (1.6.81)$$

$$\mathbf{c} = (\beta_1 h_1, \beta_2 h_2, \dots, \beta_w h_w) \quad (1.6.82)$$

where $f_1(\cdot)$ is a nonlinear activation function of the attention layer. The context vector $\mathbf{c}$ is combined with the historical target series $\mathbf{y}_w = (y_1, y_2, \dots, y_w)$:

$$\tilde{y}_i = \mathbf{w}[y_i; \mathbf{c}_i] + b, \quad 1 \leq i \leq w \quad (1.6.83)$$

Decoder LSTM and Final Prediction

The combined sequence $\tilde{\mathbf{y}} = (\tilde{y}_1, \tilde{y}_2, \dots, \tilde{y}_w)$ feeds into the decoder LSTM to produce the final prediction:

$$\hat{y}_{w+1} = f_2(\mathbf{h}'_{w-1}, \tilde{\mathbf{y}}) \quad (1.6.84)$$

where $f_2(\cdot)$ is the nonlinear activation of the decoder LSTM and $\mathbf{h}'_{w-1}$ is its hidden state from the previous time step.

1.6.7 Summary

This methodology combines established techniques in new ways to analyze the effect of three aerosol species that is BC, SO₄ and DU on near-surface temperature over the NWH region and also address temperature forecasting from aerosol data.

The multi-kernel block preserves aerosol type identity. The U-Net captures multi-scale spatial patterns. The LSTM learns temporal dependencies. The physics-informed loss encourages realistic predictions.

Each design choice has a clear justification based on either the physics of the problem or the requirements of effective learning. The architecture balances complexity with interpretability. The loss function balances accuracy with physical plausibility.

The implementation provides flexibility through hyperparameters and architectural options. This allows adaptation to different datasets and requirements. The training procedure

includes appropriate regularization and early stopping to prevent overfitting.

The resulting system provides temperature forecasts that are accurate, physically plausible, and spatially coherent. The next chapter presents experiments that validate these claims and compare the approach to alternatives.